

Task 09 - Part 02: Question Answers

C# Fundamentals - Generalization with Generics & Hierarchy Design

2. What do we mean by the Generalization concept using Generics?

Generalization in object-oriented design refers to writing code that captures a **common, reusable pattern of behavior** once, independent of the specific data type it operates on, rather than duplicating nearly identical logic for every individual type. **Generics** are C#'s language mechanism for achieving exactly this: a generic method or class (e.g., `Max<T>`, `Stack<T>`, `Helper2<T>`) is written **once**, using a placeholder type parameter `T`, and the compiler then produces a type-safe version of that logic for whatever concrete type is actually used - `int`, `string`, a custom `Employee` class, and so on - all without rewriting the underlying algorithm.

In other words, generics let you generalize the *structure* and *behavior* of a piece of code (how to reverse an array, how to swap two elements, how to compare and find a maximum) while leaving the *specific type* as an open parameter, filled in at compile time. This is a very direct example of generalization as a design principle: identifying what varies (the type) and what stays the same (the algorithm), and separating the two cleanly.

3. What do we mean by hierarchy design in real business?

In a real business application, **hierarchy design** refers to modeling how different classes relate to each other - through inheritance and composition - in a way that mirrors real organizational or conceptual structures, so the code stays organized, avoids duplication, and is easy to extend as business requirements evolve. For example, in an HR or company system, an `Employee` base class might capture properties and behavior shared by everyone (`Name`, `Id`, `Department`, `CalculateSalary()`), while more specific classes like `Manager`, `Developer`, or `Intern` derive from it and add or override behavior specific to that role (e.g., a `Manager` might override `CalculateSalary()` to add a bonus, or add a `TeamSize` property).

Designing this hierarchy well means identifying genuine "is-a" relationships (a `Manager` *is an* `Employee`) for inheritance, while using composition or association for "has-a"/"uses-a" relationships (an `Employee` *has a* `Department`, rather than inheriting from it) - exactly the kind of `Department` property added to `Employee` in the exercises above. Getting this structure right in a business system means that shared logic (like common employee properties or search behavior via `Equals()`) lives in one place, specialized behavior lives only where it's actually needed, and new employee types or business rules can be added later by extending the hierarchy rather than rewriting existing code.